Pragmatische AI Agents

LLMs, Tools & Reasoning

Sebastian Gingter
Developer Consultant
sebastian.gingter@thinktecture.com

Pragmatische AI Agents

Mit LLMs, Tools & Gedächtnis zum Ziel

- What to EXPECT
  - Overview of building blocks of an Agentic system powered by Large Language Models (LLMs)
  - Pragmatic use cases
  - Demos (mostly language independent)

- What not NOT TO EXPECT
  - ML & AI fundamentals
  - ChatGPT, CoPilot(s)
  - Deep dives into Mastra, Semantic Kernel Agent Framework, Copilot SDK, Claude SDK etc.

Pragmatische AI Agents

Mit LLMs, Tools & Gedächtnis zum Ziel

Sebastian Gingter
Developer Consultant
sebastian.gingter@thinktecture.com

Sebastian Gingter

Developer Consultant @ Thinktecture AG

- Generative AI in business settings
- AI driven Developer Productivity & Software Quality
- All things .NET

- Microsoft MVP for .NET & Azure AI Services

- sebastian.gingter@thinktecture.com
- https://www.thinktecture.com

What is an “Agent”?

Agent,
noun, countable, /ˈeɪ.dʒənt/

- someone who works secretly for the government or another organization
- a person who acts for or represents another
- a person or thing that produces a particular effect or change

Agent

- LLM with the licence to act

From LLM to Agent

Large Language Models (LLMs)

- LLMs are just a web request for us

Demo

From LLM to Agent

Recap

LLMs are stateless

LLMs are stateless

- We must add memory

Chat is just a loop now

Chat is just a loop now

- Message types
  - System
  - User
  - Assistant

- We have full control over the memory!
- We can inject and alter history if we want/need to

Tools allow interaction with the real world

Tools allow interaction with the real world

- Assistant message can carry tool calls
- New message type: tool
- Tool choice can be preset or automatic

- Provide only few tools. OpenAI docs says:
  “Aim for fewer than 20 functions at any one time”

Interlude: Model Context Protocol (MCP)

- No magic, just a standardized way of
  - providing tools to a model
  - interpreting and execution of tool calls

- Specification is still work in progress and missing a lot
  - Stateful protocol at heart
  - Authentication is an afterthought

https://julsimon.medium.com/why-mcps-disregard-for-40-years-of-rpc-best-practices-will-burn-enterprises-8ef85ce5bc9b

Tools in a loop allow agency

KI-Agenten - Einzeiler

Ein KI-Agent ist ein Programm,
das in einer Schleife zwischen dem
Reasoning mit einem LLM und dem
Ausführen von Tools wechselt,
bis es sein Ziel erreicht.

AI-Agent – One-liner

An AI-Agent is a program, which loops between reasoning with a LLM and the calling of tools, until it reaches its goal.

That’s it!

Or is it?

What do we have now?

- There are a LOT of issues with that simple approach
  - Memory grows over time (performance, cost, context window size overrun)
  - Unstructured trial and error (depending on model)
  - Functions limit what the agent can do
  - User has very little control over what happens
  - Flow is easily interrupted by external errors
  - etc…

Memory / Prompt / Context Management

- Memory growth
- Context window limits
  - Prompt Caching
  - Selective memory
  - Compacting (Summarization in intervals)

Unstructured trial & error

- ReAct – Pattern (Reasoning & Acting)
- Reasoning models do help
  - Prompt for a plan first, then continue
  - Ask to create a todo-list and follow that

Function limits

- Filter functions by context
  - Embeddings can help
- Sub-agents
  - Hand a sub-task over to agents with specific tools

Limited control

- Approval gates
  - Agent should pause before certain tasks
  - Certain tools require user-approval (outside of agent loops)
  - User can approve/deny/request changes
- Real-time monitoring
  - User sees work and can interrupt with corrections

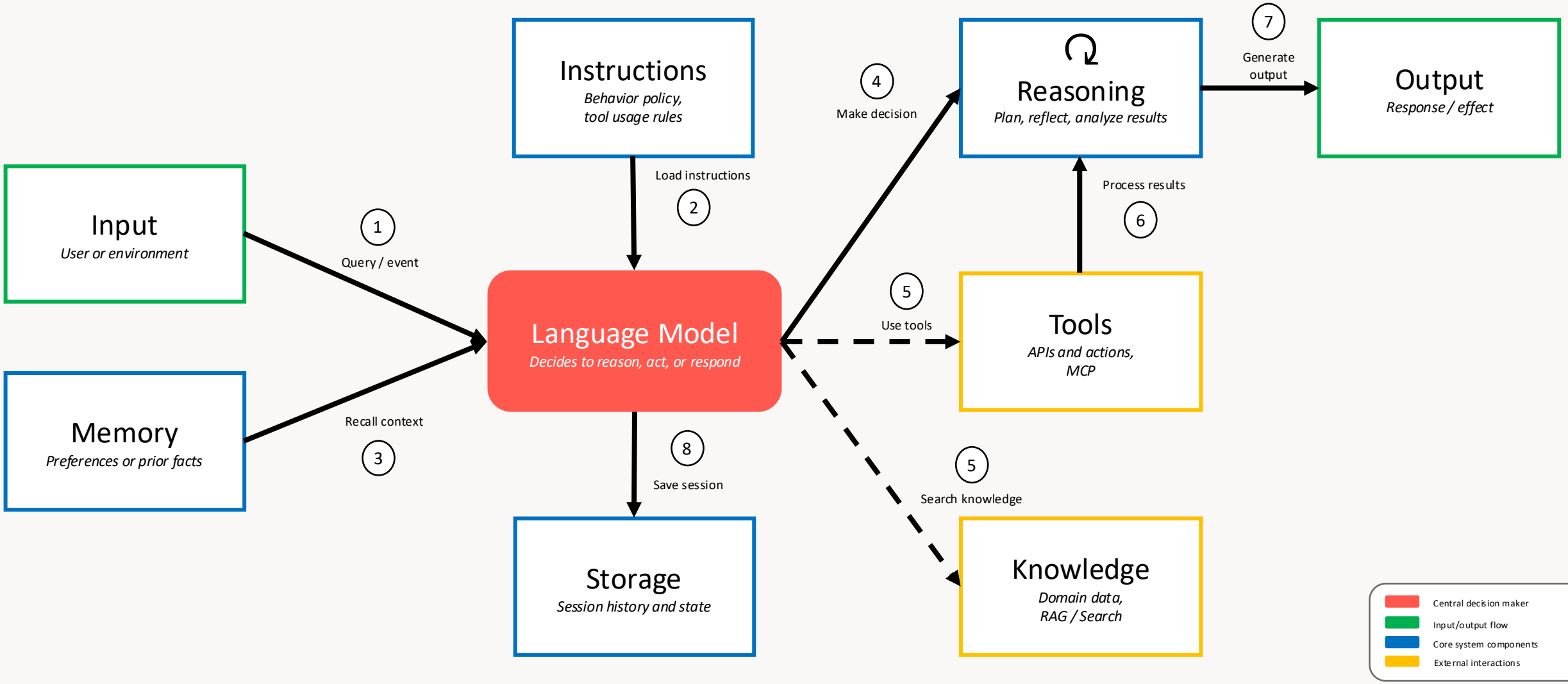
Resilience

- A working agent is a long-running process between different systems
  - treat it like that

- Agent state is serialized / persisted between roundtrips
- Partial results should be stored

Pragmatic ideas

Agents Architecture Overview



Different execution modes

- Autonomous – Agent works uninterrupted
- Step-by-step – Pause before every tool call
- Supervised - Agents actively asks when unsure
- You can switch between modes if necessary

Progress broadcasting

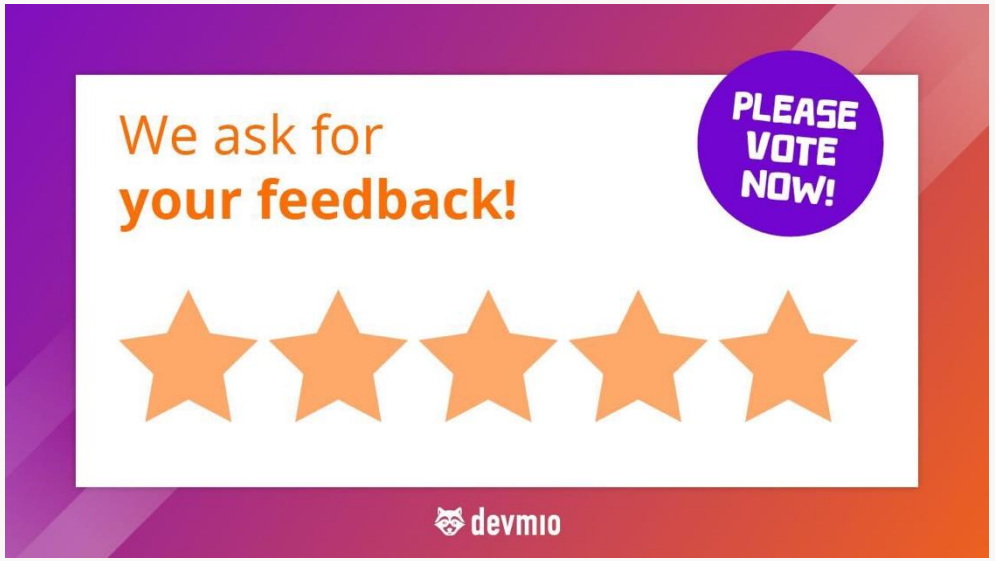
- Using of real-time / streaming APIs
- Reasoning and current steps are always visible to the user
- User can interrupt / stop the process
- User can inject (corrective) messages at every time

Demo

Example Agent

Thank you!

think tecture



Demos:

https://github.com/thinktecture-labs/basta-2026-spring-agents

Sebastian Gingter
sebastian.gingter@thinktecture.com
https://thinktecture.com/sebastian-gingter